

DAY 01: GIT & THE DEVELOPMENT WORKFLOW

Data Bytes Technologies · Java 25 & Modern Stack

Welcome. Over two weeks you'll go from individual tools to a running microservices stack. We start with **Git**, because from this morning on, *every* file we write travels through it.

// YOUR INSTRUCTOR

Mohammed Abdulrazeg

- Technical Architect / Principle Engineer / Director —
e.g. Senior Backend Engineer
- 12+ years building production JVM systems; recently
Java 25 + Quarkus microservices
- Shipped the kind of stack we build this fortnight:
REST, PostgreSQL, **Kafka**, observability

💡 Why this matters to you: I've made the mistakes you're about to make — and I'll point them out **before** they cost you an afternoon.

How I'll run the room

- Tools first, then we **build** — every concept earns its place in the running app.
- Stop me the moment something's fuzzy. A quiet question now saves an hour later.
- We learn by doing: short ✍️ *try-it-now* drills throughout, not just slides.

// THE TWO WEEKS AHEAD

This is the map. Each day is one tool; by the end they snap together into one running system.

Week 1: your toolbox

- Day 1: Git (you are here)
- Day 2: Docker & Compose
- Day 3: Java for polyglots
- Days 4–5: Java concurrency

Week 2: the running system

- Day 6: PostgreSQL (talk to it with `psql`)
- Day 7: Kafka (talk to it from the CLI)
- Days 8–10: Quarkus: we build the **app**

💡 The first week is **skills**; the second week we **build**: a small order-processing service, commit by commit. Today's Git habits are the rails that whole build runs on.

// WHY GIT, AND WHY FIRST?

You'll write code all fortnight. Where does it live, and how do many hands change it safely?

- Git is the **single source of truth** for everything we make: app, infra, docs, slides.
- It's how a team makes **many changes at once** without overwriting each other.
- It's a **time machine**: every state we've ever had is recoverable. (Today's superpower.)
- And it's how we'll **review** each other's work, where quality actually gets added.

Ten minutes on the model underneath saves you from memorising commands you don't trust.

// BY THE END OF TODAY, YOU CAN...

- explain the **Git object model** (commits, trees, blobs, refs) and why branching is free
- move confidently through the **three trees**: working directory, staging area, repository
- choose **rebase** or **merge** on purpose, and know which one is dangerous when
- **create a conflict and resolve it** without panic, and ship a clean, reviewable history
- *(stretch)* **undo almost anything**: `restore`, `reset`, `revert`, and the `reflog` safety net

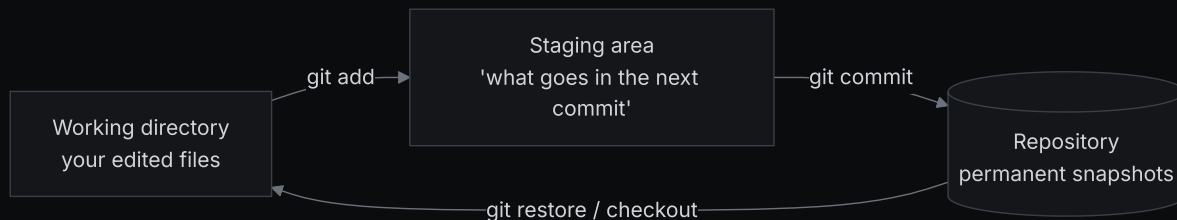
// FIRST, IN PLAIN TERMS: WHAT IS GIT?

💡 **In plain terms:** Git is a tool that takes **snapshots** of your project folder over time, and lets you move between, combine, and share those snapshots. It is not a folder of "latest files". It's the *whole history* of folders.

- A **repository** ("repo") is your project folder plus a hidden `.git/` holding every snapshot.
- It runs **on your machine**, no server needed to commit. You sync with others when you choose.
- Everything is **local and fast**, which is why branching and undoing cost almost nothing.

// THE THREE TREES YOU WORK WITH

Every Git command moves changes between **three places**. Know these three and Git stops surprising you.



The **staging area** is the one most people skip. It's a *draft of your next commit*. You choose exactly what goes in, so each commit can be one clean idea.

// INSTALL GIT BASH

Download Git Bash <https://git-scm.com/downloads>

// TRY IT NOW (3 MIN): WATCH THE THREE TREES

In this repo, make a change and watch it move through the trees:

```
echo "hello from $USER" > labs/day-01/scratch.md
git status                      # red = in working dir, not staged
git add labs/day-01/scratch.md
git status                      # green = staged, ready for the next commit
git commit -m "docs: my first scratch note"
git status                     # clean = the snapshot is recorded
```

`git status` is the command you'll run the most all fortnight. It always answers: *where are my changes right now?*

// ONE-TIME SETUP (DO THIS ONCE PER MACHINE)

```
git config --global user.name "Your Name"
git config --global user.email "you@databytes.example"
git config --global init.defaultBranch main
git config --global pull.rebase true      # keep history linear on pull (more on this later)
```

- Your **name and email** get stamped into every commit you make. Set them honestly.
- `--global` means "for all my repos"; drop it to set something for **this repo only**.

// SIGN UP FOR GITHUB

So far Git lives **only on your laptop**. GitHub is where your repo goes to be shared, backed up, and reviewed.

- Create a free account at github.com/signup — pick a username you'd put on a CV.
- Verify your email; that's all you need to create repos and push today.
- Use the **same email** you set in `git config user.email`, so your commits link to your account.

💡 Do this now. Later today, when we reach `git push`, you'll create a repo here and send your commits to it — no account, no push.

[Sign in](#)

The future of building happens together

Tools and trends evolve, but collaboration endures. With GitHub, developers, agents, and code come together on one platform.

[Sign up for GitHub](#)[Try GitHub Copilot](#)



Dashboard



githubmo ▾

Top repositories



New

Find a repository...



githubmo/lenslocked



githubmo/modern-java-25-course



githubmo/mozek.tech



githubmo/databytes.tech



githubmo/learn-go-with-test



githubmo/kanagawa-wave-jetbrains-theme



githubmo/vibe-tts

Show more

Home

Feed



Filter



can1357/oh-my-pi released

7 hours ago



v16.2.2

@oh-my-pi/pi-agent-core

Added

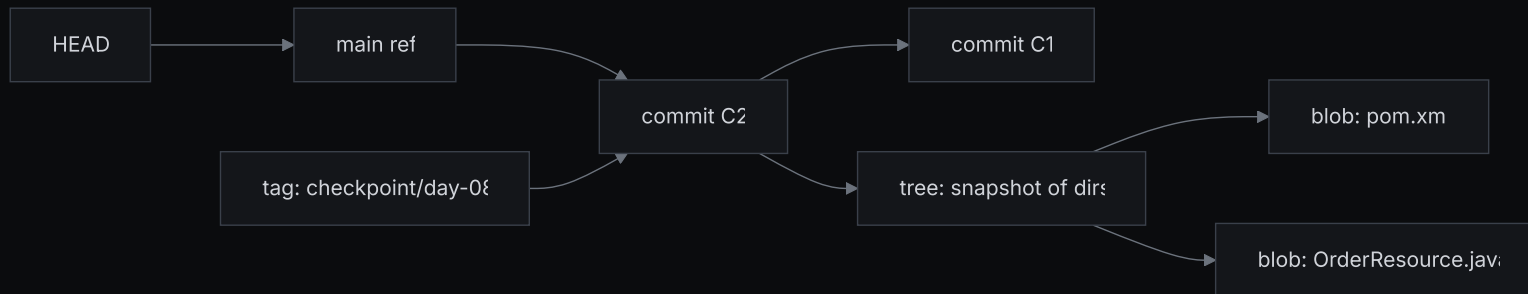
- Added optional AgentTool.matcherPaths(args) and AgentTool.matcherEntries(args) hooks to allow tools to surface target file paths and isolate file evaluations for path...

// THE OBJECT MODEL, IN PLAIN TERMS

💡 **In plain terms:** under the hood Git is a little key-value store. A **commit** is a snapshot of your whole tree plus a pointer to its parent. **Branches** and **HEAD** are just sticky notes pointing at a commit, not copies of anything.

That one fact explains everything else today: branching is free because it's one sticky note, and rebase is safe because you're only *moving* snapshots, not editing files by hand.

// ONE PICTURE: COMMITS, TREES, BLOBS, REFS



Four object types, joined into a **DAG**. The arrows are the whole story: a commit knows its parent and its tree; the tree knows its files; the labels float on top.

// TRY IT NOW (3 MIN): SEE THE OBJECTS

See the objects with your own eyes, in this repo:

```
git cat-file -p HEAD          # a commit: tree + parent + author + message
git cat-file -p HEAD^{tree}   # the tree: filenames → blob hashes
git log --oneline --graph -5  # the DAG, as labels over snapshots
```

The hashes you see *are* the addresses in the key-value store. Same content, same hash, everywhere.

// BRANCHES AND HEAD ARE JUST POINTERS

- A **branch** is a 41-byte file holding one commit hash. Creating one is instant, no copying.
- **HEAD** is "where I am right now", usually pointing at a branch, which points at a commit.
- **Switching branches** just moves **HEAD** and updates your working files to match that snapshot.

```
git switch -c feature/experiment  # new sticky note at the current commit, and move HEAD to it
git switch main                  # move HEAD back; working files snap to main's snapshot
```

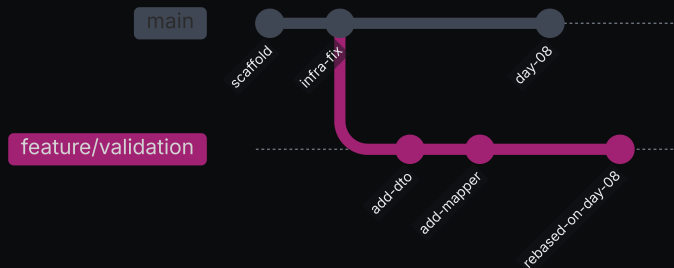
Because a branch is *one tiny pointer*, you should make them freely, one per change.

// REBASE VS MERGE: SAME GOAL, DIFFERENT HISTORY

Both combine work from two branches. They differ only in the **shape** they leave behind.

- **Merge** records exactly what happened: a merge commit with **two parents**. Honest, but a busy repo turns into a thicket of crossing lines.
- **Rebase** *replays* your commits on top of the other branch: a **linear** history that reads top-to-bottom like a story.

// A SHORT-LIVED BRANCH, REBASED ONTO MAIN



After the rebase the feature commits sit *on top of* the latest `main`, no merge node, no crossing lines. The mainline stays a single clean column.

// THE ONE RULE THAT KEEPS REBASE SAFE

💡 **In plain terms:** rebase *rewrites* your commits (new parents → new hashes). That's perfect on a **private** feature branch only you have. It's dangerous on a **shared** branch others have already based work on. You'd yank the ground out from under them.

So: **merge** when you must preserve true topology or the branch is shared; **rebase** short-lived private branches to keep the mainline linear and reviewable.

// POLISH A BRANCH BEFORE REVIEW

A reviewer should read **intent**, one coherent commit at a time, not your typo-fix archaeology.

```
git switch -c feature/validation    # branch: a cheap, movable sticky note
git commit --fixup HEAD~1          # mark a tweak to fold into an earlier commit
git rebase -i --autosquash main     # reorder, reword, fold fixups, one story
```

`--fixup` plus `--autosquash` squash each little fix **into the commit it belongs to**, automatically. You ship the story you *meant* to write, which is exactly how this repo's history is built.

// TRY IT NOW (4 MIN): FOLD A FIXUP INTO ITS COMMIT

Make two commits, then fold the second into the first. Leave one clean commit behind:

```
git switch -c feature/$USER-polish
printf '%s\n' "$USER learned something" >> labs/day-01/scratch.md
git add -A && git commit -m "docs: add my scratch note"
printf '%s\n' "...and fixed a typo" >> labs/day-01/scratch.md
git commit -am 'fixup!' --fixup HEAD      # mark it as a fixup
git rebase -i --autosquash main          # save+close the editor to apply
git log --oneline -3                     # one coherent commit, not three
```

// 🖐️ EVERYONE WITH ME?

You should, right now, be able to:

- name the **three trees** and what `git add` / `git commit` move between them
- read a real **commit object** and name its tree and parent
- say **why** rebase is safe on a private branch and risky on a shared one
- turn **two commits into one** with `--fixup` and `--autosquash`

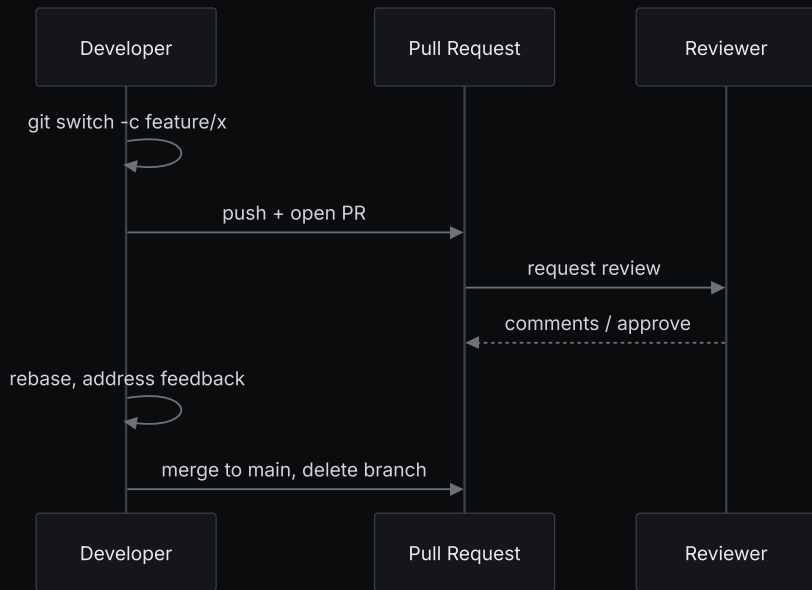
If any of those is fuzzy, flag it now. The undo, conflict, and lab sections build directly on these.

// BRANCHING STRATEGY: SHORT-LIVED, ALWAYS

Long-lived branches **rot**: the longer a branch lives, the further it drifts, the worse the merge.

- This course models **trunk-based development**: branch, make **one** coherent change, open a PR, merge within a day or two.
- Releases are marked with **tags** (`checkpoint/day-NN`), not permanent release branches.
- Small branches mean small reviews, and small reviews actually get done well.

// THE PULL-REQUEST LOOP



The branch is temporary scaffolding. The **review** is where quality is added. Then the branch is deleted and the story lives on in `main`.

// CODE REVIEW FOR INTENT

A good pull request tells a **story**.

- A clear **title**, a body explaining **why** (not just what), and commits that each do **one thing**.
- Review for **intent and correctness**, not only style: *does this do what it claims, and will the next person understand it in six months?*
- In week 2, every change to the reference app is small enough to review **in one sitting**, on purpose.

// CONFLICTS: NOT AN ERROR, A QUESTION

When you get a conflict, dont panic. It's Git saying: *two changes touched the same line. Which do you want?*

```
<<<<<< HEAD
your version of the line
=====
their version of the line
>>>>>> feature/theirs
```

You edit the file to the answer you want, delete the markers, `git add`, and continue. That's it, no magic, no lost work.

// TRY IT NOW: MAKE A CONFLICT, THEN RESOLVE IT (5 MIN)

Two branches touch the **same line**, then merge one into the other:

```
git switch main
git switch -c feature/conflict-a
printf 'shared line - version A\n' >> labs/day-01/scratch.md
git commit -am "scratch: version A"

git switch main && git switch -c feature/conflict-b
printf 'shared line - version B\n' >> labs/day-01/scratch.md
git commit -am "scratch: version B"

git switch feature/conflict-a && git merge feature/conflict-b  # CONFLICT
# edit scratch.md to the line you want, remove the markers:
git add labs/day-01/scratch.md && git commit
```

// DON'T COMMIT WHAT SHOULDN'T BE COMMITTED

Git happily records **everything** you `add`, including things that must never leave your machine.

- **Secrets:** passwords, API keys, `.env` files. Once committed and pushed, treat them as **leaked**, even if you delete them later (they live in history).
- **Build noise & junk:** `target/`, `node_modules/`, `.DS_Store`, logs.
- A `.gitignore` file lists patterns Git should never track:

```
# .gitignore
target/
*.env
.DS_Store
```

// MONO-REPO VS MULTI-REPO

Where does *all this code* live? This course makes one deliberate choice.

- **Mono-repo (this course):** app, infra, docs, slides, diagrams in **one** repo, versioned together. A schema change, its migration, and its docs land in **one PR**, atomically.
- The cost: you need tooling that understands a polyglot tree, which is what our `Taskfile` gives.
- **Multi-repo** suits independent teams on separate release cadences, at the cost of coordinating cross-service changes across many PRs.

// COMMON TRAPS

- **Committing secrets:** a `.env` or key in history is leaked forever. Use `.gitignore`; if it happens, rotate the secret immediately.
- **Rebasing a shared branch:** you rewrite history others rely on. Rebase only your own private work; force-push only your own PR branch, with `--force-with-lease`.
- **Panicking at a conflict:** it's a question, not a crash. Edit, remove markers, `git add`, continue. `git merge --abort` is always your escape hatch.
- **Pile-of-fixups history:** "wip", "fix", "address review" × 8. Fold them with `--autosquash` *before* asking for review.

// RECAP, IN PLAIN TERMS

Today you stopped *using* Git and started *understanding* it:

- changes flow through **three trees**: working dir → staging (`add`) → repository (`commit`)
- a **commit** is a snapshot; a **branch** is a sticky note, so branching is free
- **rebase** rewrites *your* commits for a clean line; **merge** records true topology. Pick on purpose
- a **conflict** is a question you answer by editing; short-lived branches + review are where quality lives
- (*stretch*) you can **undo almost anything**: `restore` , `reset` , `revert` , and `reflog` has your back

// 🚀 STRETCH GOAL: UNDO — NOTHING IS REALLY LOST

Everything from here is **optional today**. It's the section that kills the fear of breaking things — teach it live if there's time, or send fast finishers here during the lab.

// UNDO, IN PLAIN TERMS: NOTHING IS REALLY LOST

💡 **In plain terms:** almost every "oh no" in Git is recoverable, because committing only *adds* snapshots; it rarely destroys them. The trick is knowing *which* undo you want.

- Threw away the **wrong edit** (not committed yet)? → `git restore`
- Want to **move the branch pointer** back? → `git reset`
- Already **shared** the bad commit and need a safe undo? → `git revert`
- **Truly lost** a commit? → `git reflog` finds it

// **RESTORE** **AND** **RESET** : UNDO THE NOT-YET-SHARED

```
git restore labs/day-01/scratch.md    # discard unstaged edits to a file (back to last commit)
git restore --staged scratch.md        # unstage, but keep your edits

git reset --soft HEAD~1                # undo the commit, KEEP changes staged
git reset --mixed HEAD~1               # undo the commit, keep changes unstaged (the default)
git reset --hard HEAD~1                # undo the commit AND throw away the changes (careful!)
```

--soft / **--mixed** / **--hard** is just **how far back the changes travel**: into staging, into your working files, or gone.
Map them onto the three trees.

// REVERT AND REFLOG : THE SHARED-SAFE UNDO AND THE SAFETY NET

- `git revert <hash>` makes a *new* commit that undoes an old one. History is preserved, so it's safe on shared branches, the inverse of `reset`.

```
git revert HEAD          # "undo the last commit" as a new, honest commit
```

- `git reflog` is Git's private logbook of *every* place `HEAD` has been, even commits you "lost" by resetting. Your ultimate undo.

```
git reflog              # find the hash you thought was gone
git reset --hard <hash> # ...and jump right back to it
```

// TRY IT NOW (4 MIN): LOSE A COMMIT, THEN RESCUE IT

Prove to yourself that a "lost" commit isn't gone:

```
git switch -c feature/$USER-undo
echo "important work" >> labs/day-01/scratch.md
git commit -am "feat: important work"
git log --oneline -1          # note the message

git reset --hard HEAD~1      # the commit "disappears"
git log --oneline -1         # ...it's gone from the branch

git reflog                   # find your commit's hash in the logbook
git reset --hard <that-hash> # and bring it back
git log --oneline -1         # restored!
```

// TODAY'S LAB

✍ **Practise the full workflow:** branch off `main`, commit cleanly through the three trees, deliberately create and resolve a conflict, rebase to a linear history, and review a partner's change for intent.

Brief: `labs/day-01/` · You practise on **real history**, this repo, no starter scaffold.

Acceptance: one coherent rebased commit, a conflict resolved by hand, a reviewed change merged and the branch deleted, and a `git log --graph --oneline` that reads as a clear story.

Stretch: rescue a "lost" commit with `reflog` (see the stretch section).

// TAKEAWAY

Treat history as something the **next reader** must understand. Keep branches short, changes small, and let the **review**, not the branch, be where quality is added. And if you reach the stretch section: with `reflog` behind you, you can experiment **fearlessly**.

Tomorrow, Day 2: Docker & Compose. We make your environment reproducible: one command spins up the same Postgres, Kafka, and tools for everyone in the room, the foundation the week-2 app runs on.